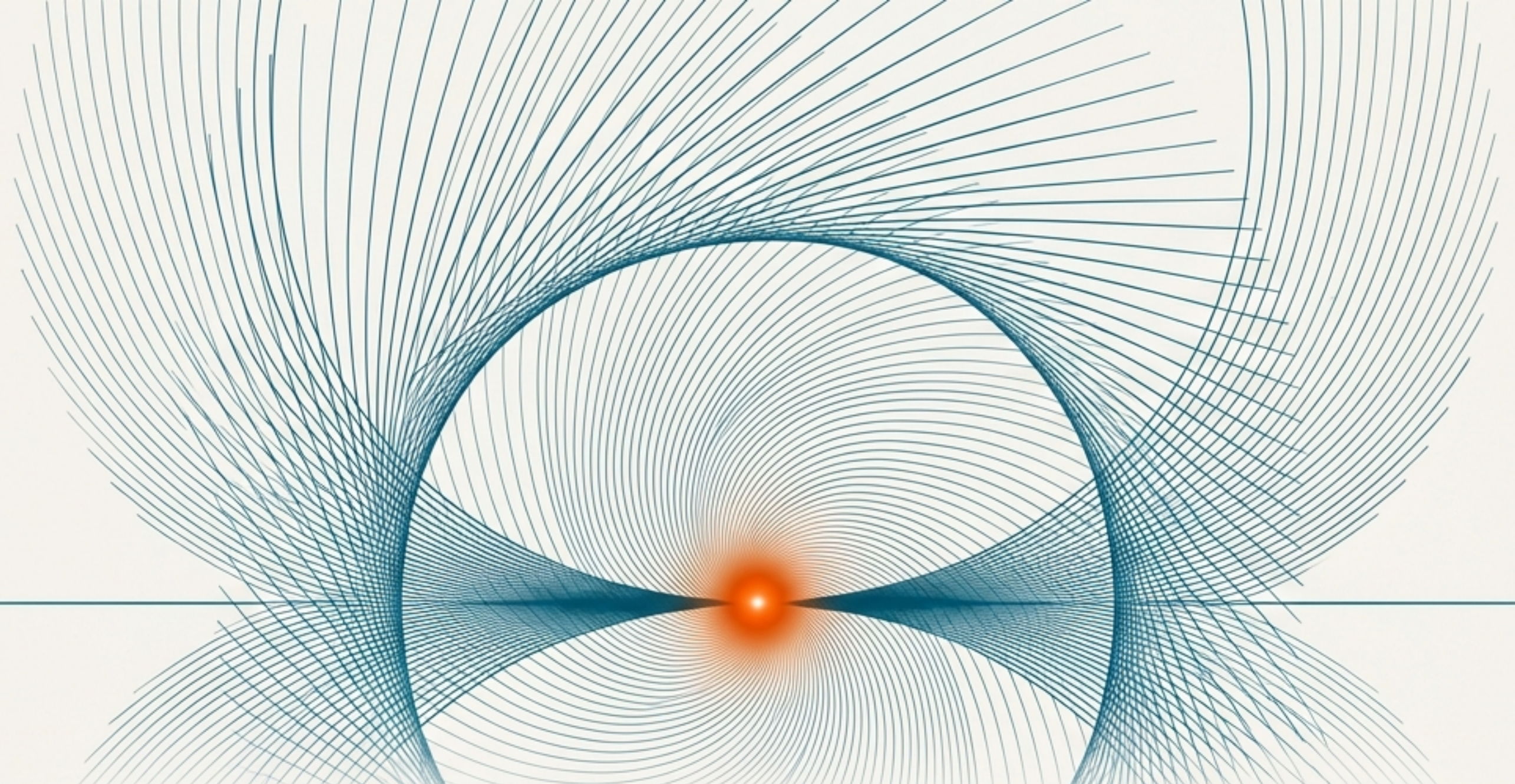




# EL ARTE DE LA APROXIMACIÓN

Fundamentos y Algoritmos del Análisis Numérico

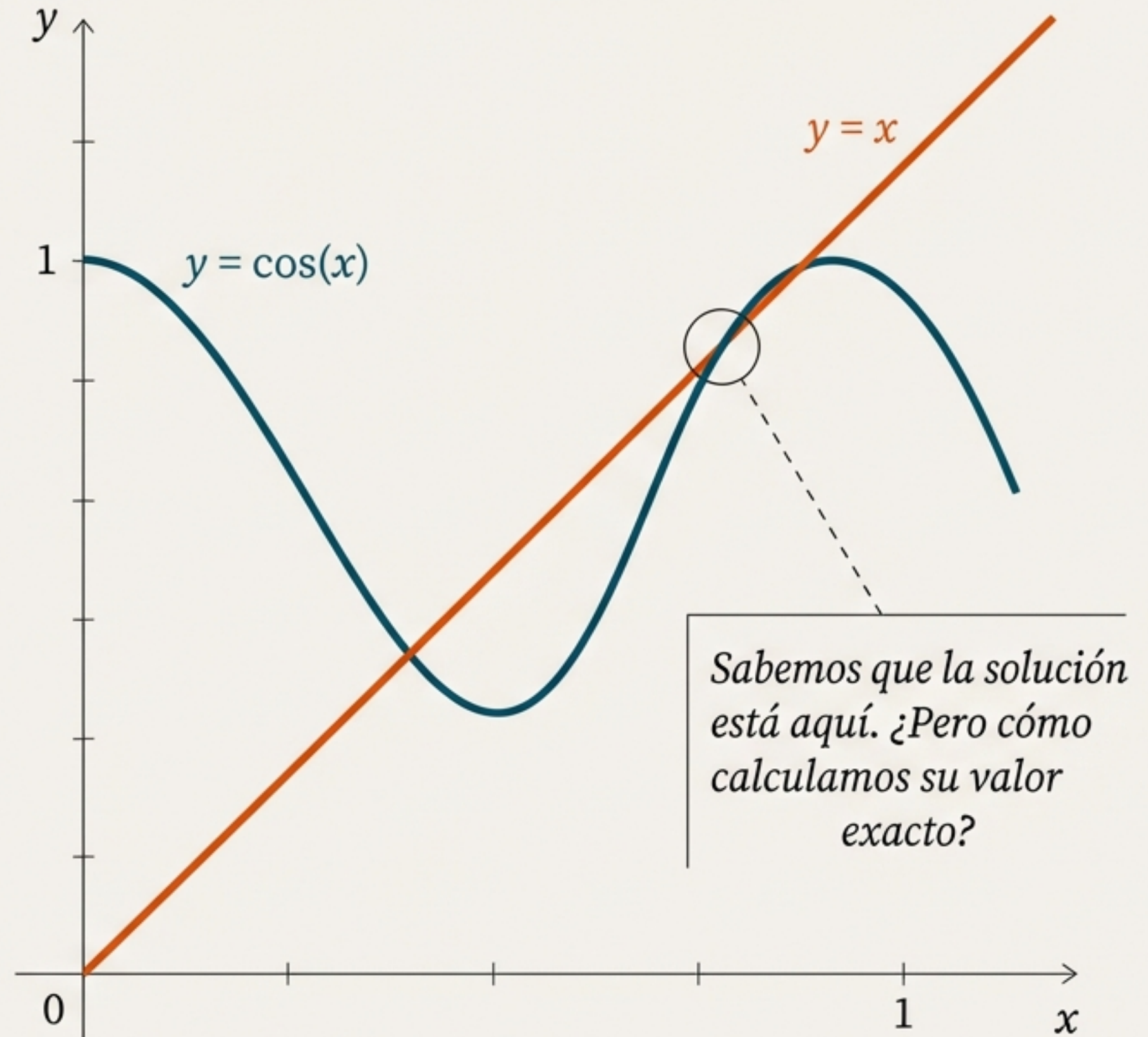
---

Una guía para construir soluciones precisas cuando las fórmulas exactas no existen.

## EL LÍMITE DE LO EXACTO: ¿POR QUÉ APROXIMAR?

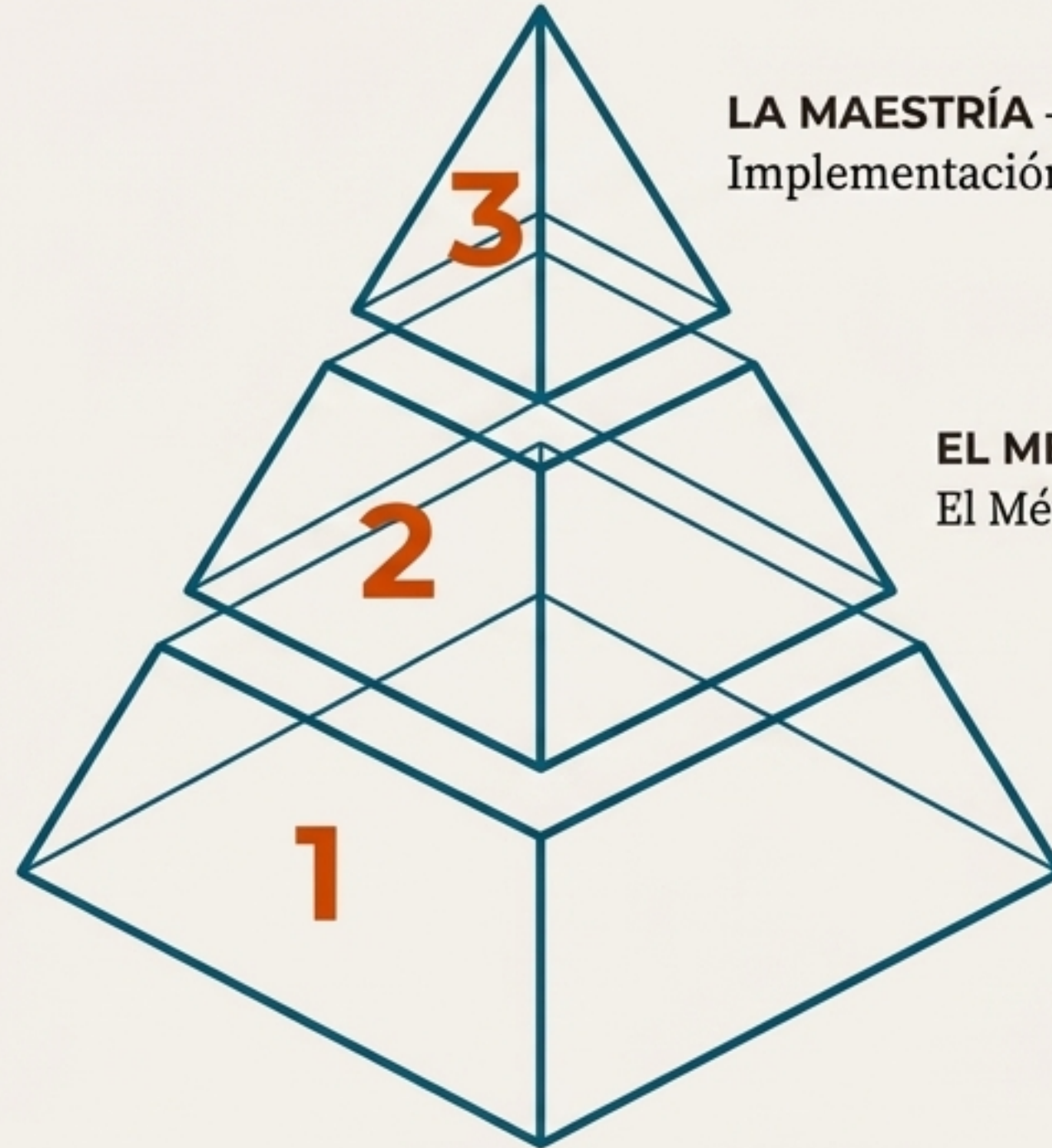
Muchas ecuaciones fundamentales en ciencia, ingeniería y finanzas no tienen una solución analítica simple. No podemos 'despejar la  $x$ '. Sabemos que existe una solución, pero no podemos escribirla.

Consideremos la ecuación  $\cos(x) = x$ . Es simple de escribir, pero imposible de resolver algebraicamente. El análisis numérico nos da las herramientas para encontrar esa ' $x$ ' con la precisión que necesitamos.



# NUESTRO PLAN: DE LA BASE A LA CIMA

Construiremos nuestro conocimiento en tres niveles. Cada uno se apoya en el anterior, creando una estructura sólida para entender y aplicar los métodos numéricos.



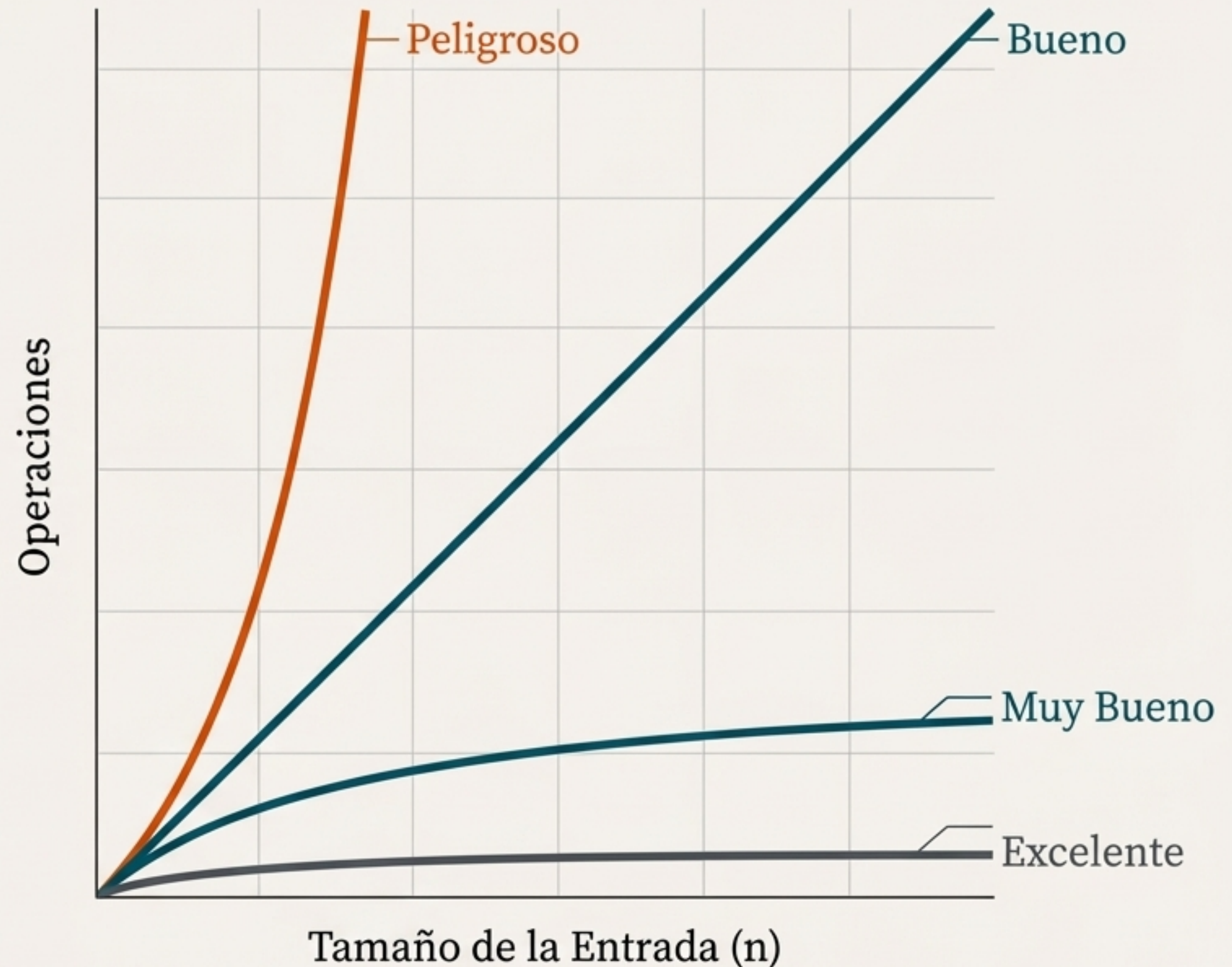
**LA MAESTRÍA** - La aplicación y el futuro:  
Implementación, cotas de error y exploración con IA.

**EL MÉTODO** - La herramienta de solución:  
El Método de Bisección.

**FUNDAMENTOS** - El lenguaje de la  
precisión: Notación Big O, Convergencia  
y Funciones de Lipschitz.

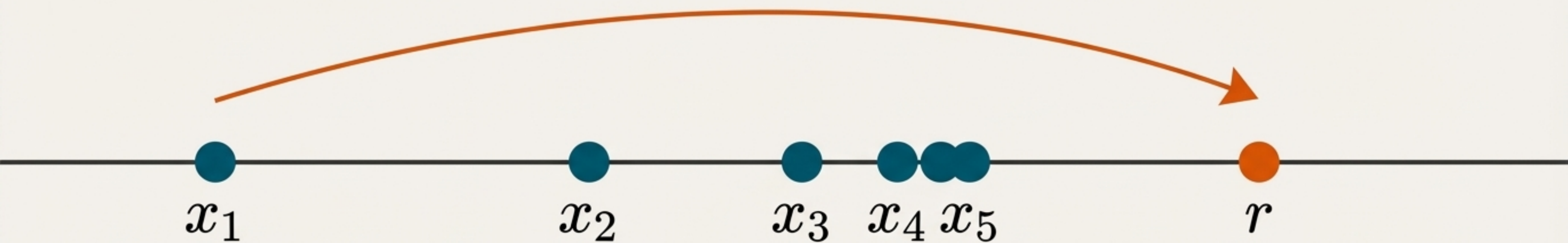
# FUNDAMENTO 1: NOTACIÓN BIG O, EL LENGUAJE DE LA EFICIENCIA

La notación Big O no mide el tiempo en segundos, sino cómo responde un algoritmo al crecimiento de los datos. Nos permite comparar la eficiencia de diferentes métodos de forma abstracta y universal. ¿Es una solución escalable o se colapsará con datos reales? Big O tiene la respuesta.



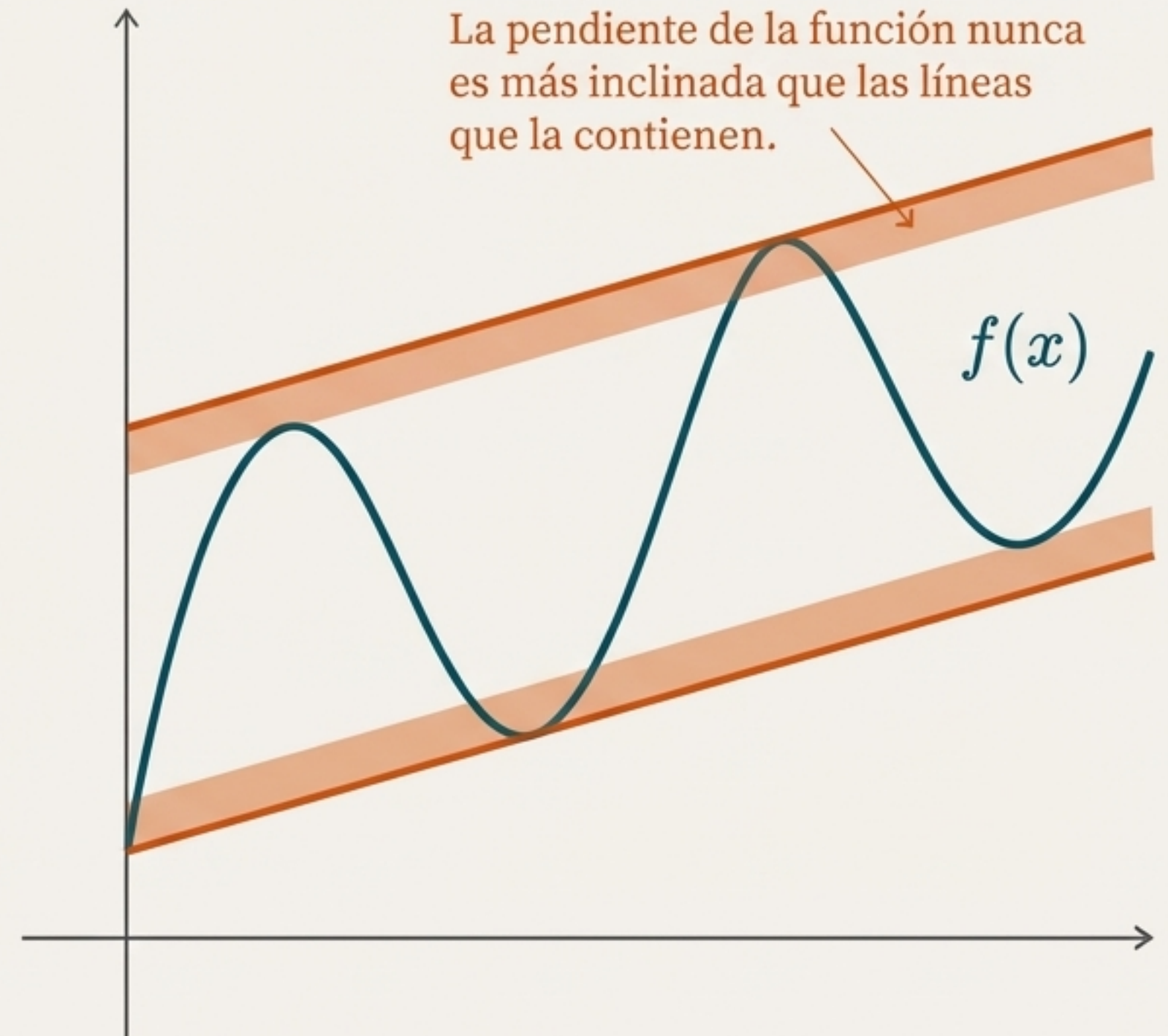
# FUNDAMENTO 2: CONVERGENCIA, EL CAMINO HACIA LA RESPUESTA

Un método numérico es útil solo si sus aproximaciones sucesivas se acercan a la solución real. Este concepto se llama convergencia. Es la garantía matemática de que nuestro esfuerzo iterativo tiene un destino y no estamos perdidos en el caos computacional.

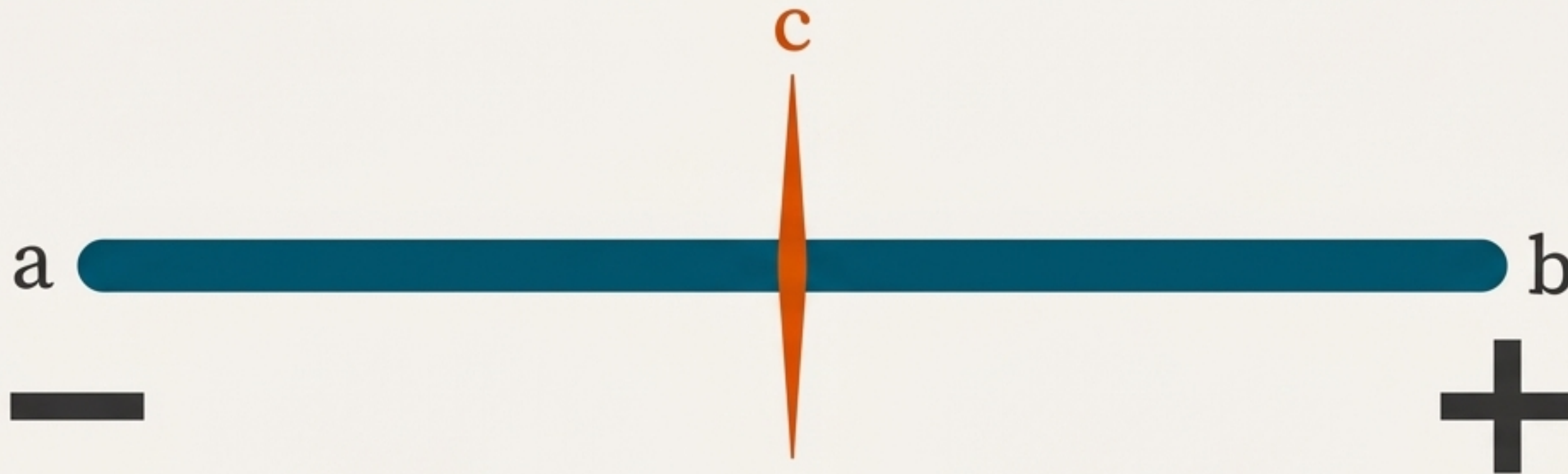


## FUNDAMENTO 3: FUNCIONES DE LIPSCHITZ, LA GARANTÍA DE ESTABILIDAD

La condición de Lipschitz asegura que una función no cambia de manera demasiado abrupta. Limita la 'pendiente' máxima de la función, garantizando que el comportamiento en un punto nos da una idea confiable del comportamiento cercano. Esta predictibilidad es crucial para que muchos algoritmos de optimización y búsqueda de raíces funcionen correctamente.

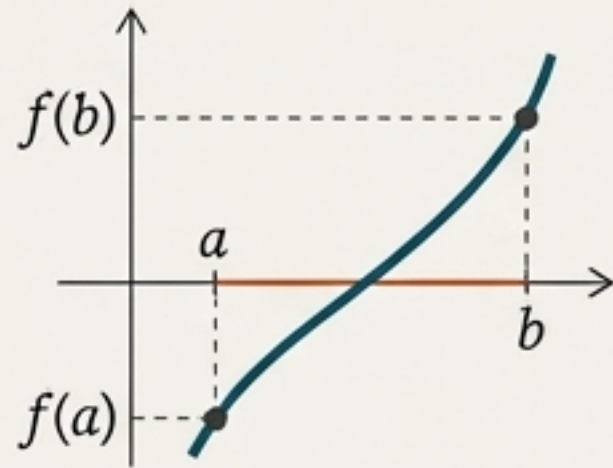


# EL MÉTODO DE BISECCIÓN: DIVIDIR PARA CONQUISTAR



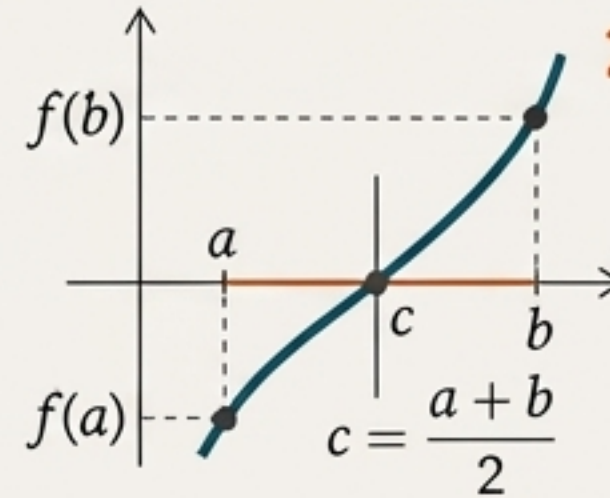
Construido sobre nuestros fundamentos, el método de bisección es un algoritmo robusto e intuitivo para localizar raíces. Su lógica se basa en una idea simple pero poderosa, garantizada por el Teorema del Valor Intermedio: Intermedio: dividir el intervalo de búsqueda a la mitad en cada paso.

# LA LÓGICA DEL ALGORITMO EN 4 PASOS



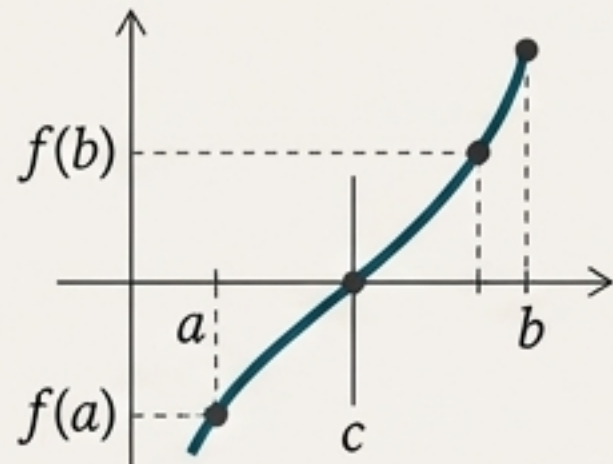
## 1. DEFINIR INTERVALO

Empezamos con un intervalo  $[a, b]$  donde estamos seguros de que existe una raíz.



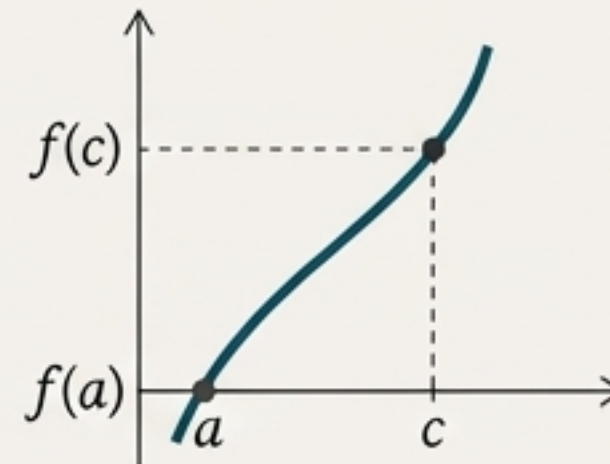
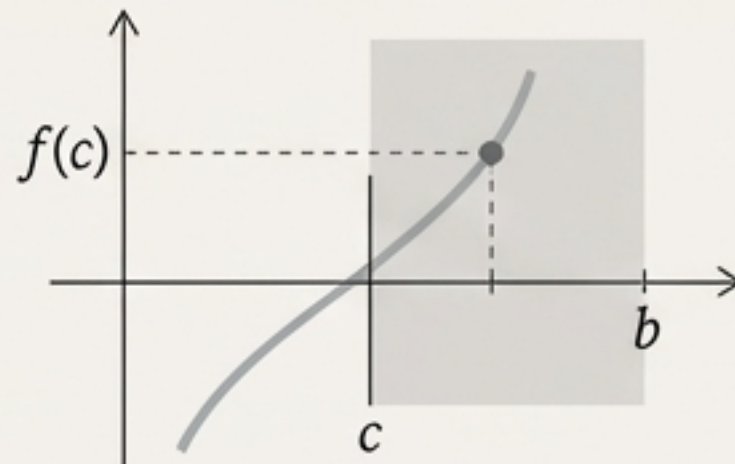
## 2. CALCULAR PUNTO MEDIO

Encontramos el punto medio exacto del intervalo.



## 3. EVALUAR Y DESCARTAR

Evaluamos el signo de  $f(c)$  y descartamos la mitad del intervalo que no contiene la raíz.



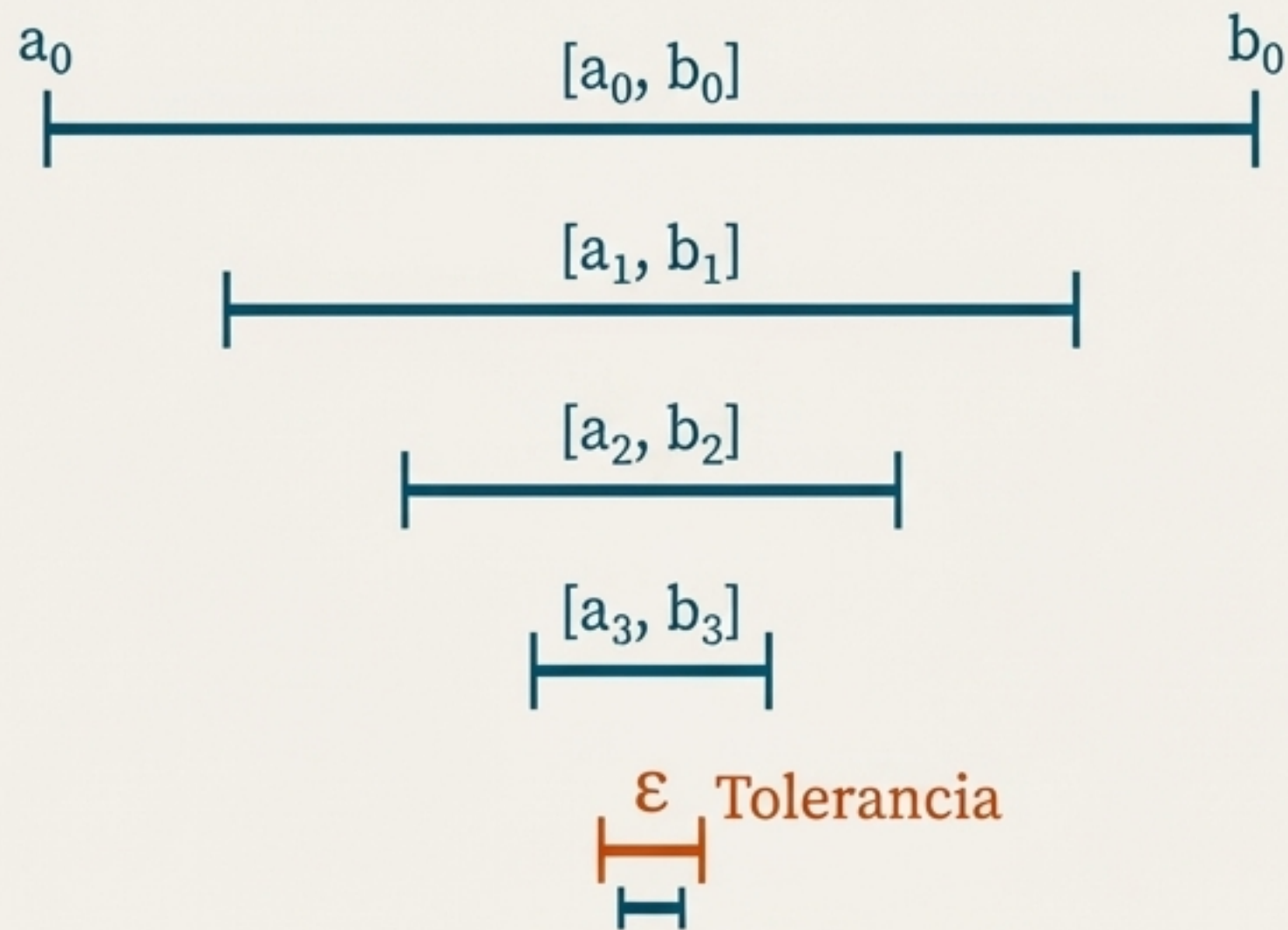
## 4. REPETIR

Repetimos el proceso con el nuevo intervalo,

Repetimos el proceso con el nuevo intervalo, reduciendo la incertidumbre a la mitad en cada iteración.

# PRECISIÓN Y CONTROL: ¿CUÁNDO NOS DETENEMOS?

Un algoritmo infinito no es útil. Necesitamos criterios de parada técnicos para detener la ejecución cuando la aproximación es “suficientemente buena”.



## Criterio 1: Tolerancia (Precisión)

Detenerse cuando la longitud del intervalo  $(b-a)$  es menor que una tolerancia predefinida  $\epsilon$  (ej.  $10^{-6}$ ). Esto garantiza la precisión de la raíz.

$$|p_n - p| \leq \frac{b - a}{2^n}$$

## Criterio 2: Límite de Iteraciones

Detenerse después de un número máximo de iteraciones  $N$  para evitar bucles infinitos si algo sale mal.

# DE LA TEORÍA AL CÓDIGO: LA IDEA EN ACCIÓN

La elegancia del método de bisección se refleja en la simplicidad de su implementación. A continuación, un pseudocódigo que encapsula la lógica que hemos descrito.

```
1  función biseccion(f, a, b, tolerancia):  
2      mientras (b - a) > tolerancia: [ Criterio de parada por precisión ]  
3          c = (a + b) / 2 [ 2. CALCULAR PUNTO MEDIO ]  
4          si f(c) == 0: [ Condición de parada ideal ]  
5              retornar c  
6          si f(a) * f(c) < 0: [ 3. EVALUAR Y DESCARTAR ]  
7              b = c  
8          sino:  
9              a = c  
10     retornar (a + b) / 2
```

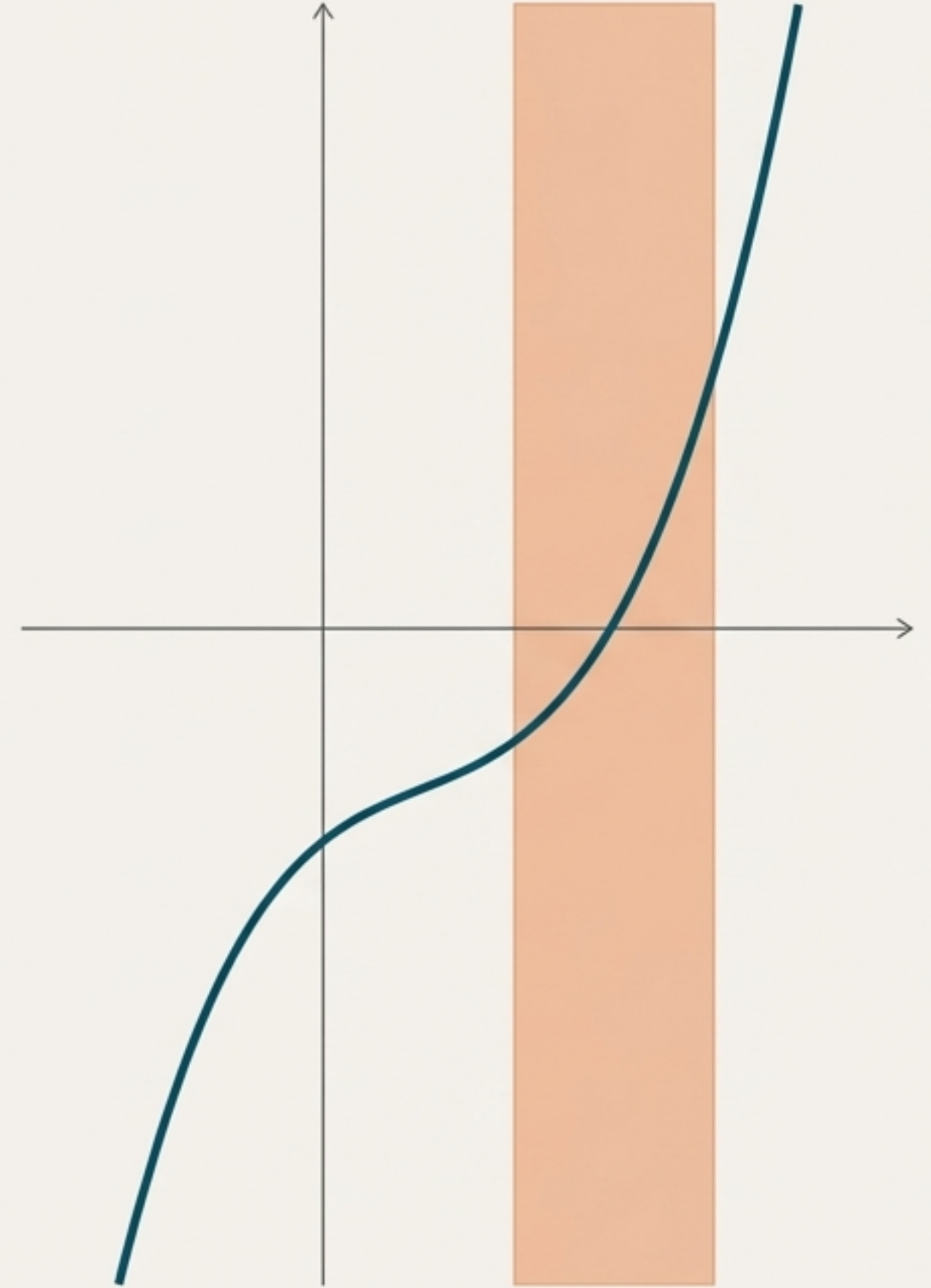
# DESAFÍO PRÁCTICO: ENCUENTRA LA RAÍZ

La mejor forma de aprender es haciendo.  
Implementa el método de bisección para  
resolver el siguiente problema.

**El Problema:** Encuentra la raíz de la función  
 $f(x) = x^3 + 4x^2 - 10$  en el intervalo  $[1, 2]$ .

## Requisitos:

1. Usa una tolerancia  $\varepsilon = 10^{-5}$ .
2. Calcula cuántas iteraciones  $n$  se necesitarán teóricamente para alcanzar esta tolerancia usando la fórmula de la cota de error.



# EL FUTURO: TU COPILOTO DE IA PARA PROFUNDIZAR

Has dominado los fundamentos. Ahora, usa la inteligencia artificial como una herramienta para explorar más allá. Trata a los modelos de lenguaje como un colaborador experto para acelerar tu aprendizaje.

## **\*\*Prompts Sugeridos para tu IA:**

Compara la velocidad de convergencia lineal del método de bisección con la convergencia cuadrática del método de Newton-Raphson. Genera un gráfico comparativo.

Genera un script en Python que visualice las primeras 5 iteraciones del método de bisección para la función  $f(x) = e^x - 2$  en el intervalo  $[0, 1]$ .

Explica en qué escenarios el método de bisección podría ser preferible a métodos más rápidos como el de la secante, y viceversa.

# TU CAJA DE HERRAMIENTAS DEL ANALISTA NUMÉRICO



## Notación Big O

Para medir y comparar la eficiencia fundamental de los algoritmos.



## Convergencia

La garantía de que nuestras aproximaciones se dirigen al resultado correcto.



## Método de Bisección

Un algoritmo robusto, simple y siempre confiable para encontrar raíces.



## Implementación y Precisión

La habilidad de traducir teoría en código funcional y controlar el error.



**LA PRECISIÓN NO ES UN  
PUNTO DE PARTIDA.  
ES UN DESTINO QUE SE CONSTRUYE.  
SE CONSTRUYE.**

# PARA SEGUIR EXPLORANDO

El análisis numérico es un campo vasto y fascinante. Aquí tienes algunos recursos clásicos para continuar tu aprendizaje.



**Libro:** *Análisis Numérico* por R.L. Burden y J.D. Faires - La referencia estándar en el campo.



**Libro:** *Numerical Recipes* por W.H. Press et al. - Un enfoque práctico con énfasis en la implementación.



**Online:** Explora implementaciones de estos y otros algoritmos en librerías como ``SciPy`` (Python) o ``Numerical.js`` (JavaScript).